

Macro Copilot.

An LLM-orchestrated research platform for macro funds, built on a typed substrate of validated quantitative primitives. Plain-English questions in, deterministic and reproducible analyses out.

COMPOSABLE

Cross-asset by construction

ITERATIVE

Follow-ups extend the prior DAG

REPRODUCIBLE

Byte-identical at six months

AUDITABLE

Methodology in version control

01 The problem

Macro hedge funds run on a specific human workflow. A portfolio manager poses an investment question, an analyst translates it into a sequence of analyses, the PM reads the output and asks a follow-up. The PM's edge is which questions get asked next. The analyst's value is executing those questions correctly and quickly. **Today this loop is bottlenecked at every stage.**

TOOL	WHAT IT CAN DO	WHERE IT BREAKS
Bloomberg / Refinitiv	Single-instrument queries: yields, spreads, levels, curves.	Cannot compose. "2s10s" works; "2s10s conditional on QT regimes, regressed against credit, rolling 60-day" does not.
Excel + Python notebooks	Compose anything; run on the analyst's own logic.	Write-once, audit-impossible, single-laptop. Reproducing a six-month-old analysis is a project, not a query.
Internal quant platforms (at large multi-strategy funds)	Compose, scale, and reproduce inside the firm.	Hundreds of millions of dollars and decades of engineering. Methodology stays locked inside the firm.
General-purpose LLMs (ChatGPT, Claude, Gemini)	Talk fluently about the question.	Cannot execute on your data. Cannot guarantee numerical reproducibility. Hallucinate as soon as the question gets specific.

The gap

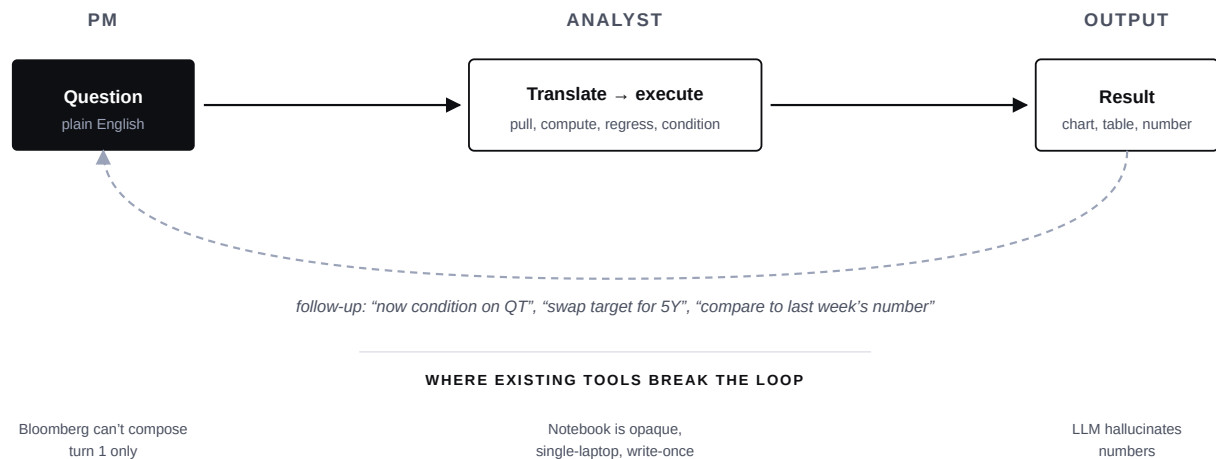
A system that does what an excellent macro analyst does: translate a PM's natural-language question into deterministic, lineage-tracked, reproducible quantitative work, iterate on follow-ups in plain English, and stay numerically honest under arbitrary composition. That is the system we are building.

WHY THIS MATTERS NOW

LLMs have made the front door of quant research conversational. The back door (the data substrate, the methodology, the lineage) has not changed. A copilot without a typed substrate is a confident hallucination machine. A typed substrate without a copilot is another internal platform. Macro Copilot is the bridge.

02 The core insight: the loop is the product

A PM does not ask one question. A PM asks a question, sees the answer, and asks a sharper one. **The value-add is in the third, fourth and fifth question, not the first.** Every existing tool optimises the first question and breaks the loop somewhere between turn two and turn three.



The macro analyst loop. Existing tools optimise the forward arrow and abandon the dotted return arrow.

What “the loop is the product” means in practice

An iterative copilot is not a chat wrapper around a tool. It is a system whose internal representation of **turn N** includes the executed graph from turn N-1, so that follow-ups *extend* the prior analysis rather than restart it. Three behaviours fall out of this:

SLOT REBINDING

“Now do that with 5Y instead of 2Y” rebinds one slot on the same workflow template; the rest of the DAG is reused unchanged.

GRAPH EXTENSION

“Now also condition on QT regimes” stitches an additional branch onto the cached DAG; only the new branch is executed.

CROSS-TURN ARITHMETIC

“Compare to the previous result” is a one-operator combination of the two cached terminal artifacts. No re-fetch, no re-compute.

BRANCHING STATE

“Save this and try a variant” forks conversational state. Six variants are six addressable artifacts the PM can compare side-by-side.

STRATEGIC POINT

Tier 1 of the reasoning stack (templates) wins the first turn. Tier 2 (operator DAGs) and Tier 3 (open generation) are what win turns three through ten. **The substrate is what makes turns three through ten possible without an engineer in the loop.**

03 What we are building

An analyst's day, today: pull the data, build the model, validate the statistics, hand the chart to the PM, take the follow-up, repeat. Most of the cost lives in the round-trips. Macro Copilot collapses that loop. The analyst defines the analytical building blocks once, in code with version control. The PM types a question; the system picks a pre-validated workflow, runs it deterministically against a multi-asset time-series database, and streams the answer back with its full computation behind it. Follow-ups extend the same work instead of starting over.

What changes for the user

TURNAROUND IN MINUTES, NOT HOURS

Questions that take an analyst a working day to pull, model, and present resolve to a single prompt. The bottleneck moves from execution to question-asking, where the PM's actual edge is.

COVERAGE OF THE FULL MACRO BOOK

Rates, FX, credit, commodities, equities, vol. Single-instrument or cross-asset, conditional, lagged, regime-switching. Any question that can be expressed as a combination of validated primitives is in scope, with no new engineering work.

FOLLOW-UPS BUILD ON THE PRIOR TURN

"Now condition on QT regimes" adds a branch to the analysis just run, rather than recomputing from scratch. The conversation carries the work forward; turn three uses turn two's result.

AUDIT TRAIL ON EVERY NUMBER

Every chart ships with a click-through to the data, the methodology, and the operator graph behind it. PMs and risk teams audit by inspection, not by emailing the analyst for a notebook.

What the system is not

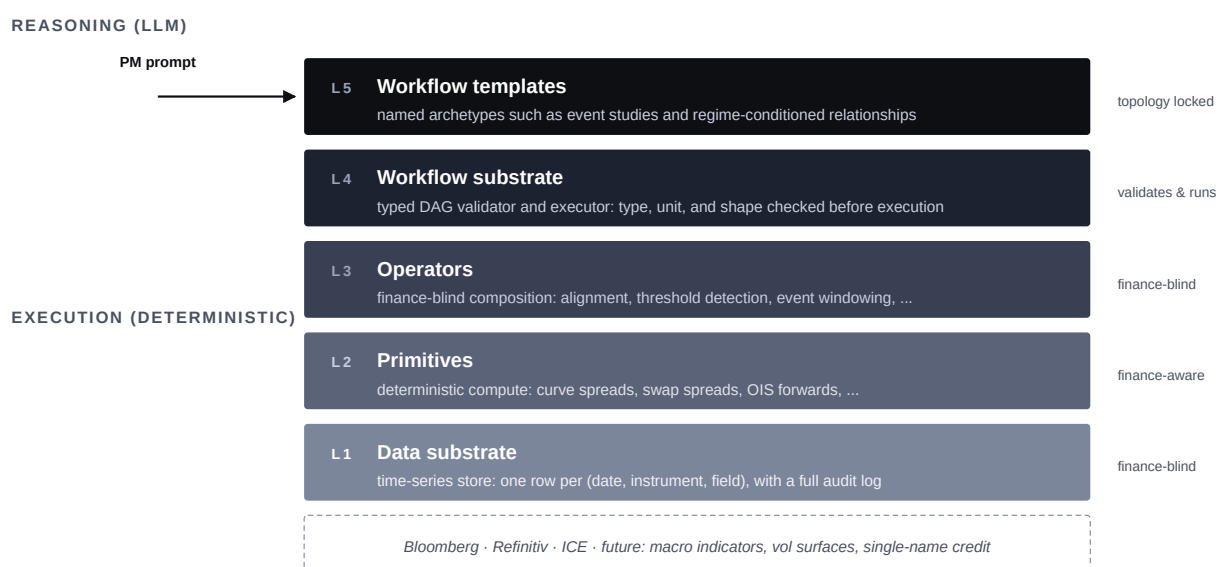
It is not "a better Bloomberg": terminals answer questions about instruments; the copilot answers questions about *relationships*. It is not "an LLM with tool calls": arbitrary tool calls produce numbers no one can defend. It is not a black-box quant platform: every methodology choice is in version-controlled configuration and every result has a lineage chain. **It is the analyst, with the audit trail of an internal quant platform and the conversational access of a chat product.**

HOW THE SPEED-UP IS NOT A CORNER-CUT

The compression from hours to minutes does not come from cutting analytical corners. Every methodology choice an analyst would normally make (which sample window, which adjustment convention, which missing-data policy) is encoded once in the substrate, reviewed once, and reused under every subsequent question. The PM gets minutes-to-answer; the firm keeps the audit trail of work done in code.

04 Architecture: five layers, strict invariants

Macro analysis touches an effectively unbounded set of questions, but those questions decompose into a finite vocabulary: data, the calculations applied to data, and the ways those calculations chain together. The architecture mirrors that structure. Five strict layers separate concerns, and each layer's rules are enforced by the layer below. The LLM works at the top, choosing among components that already exist; it never invents new math. **The flexibility comes from the combinatorics: a finite catalogue of building blocks generates the open-ended question space the PM actually asks against.**



Five layers. The LLM sits above L5 and only emits typed graphs that the layers below admit. L1–L4 are deterministic.

LAYER SEPARATION IN ONE SENTENCE

L1 holds raw data. L2 turns data into market quantities (yields, spreads, betas). L3 transforms those quantities. L4 chains transformations together safely. L5 holds question patterns the firm has already vetted. The LLM only chooses among L5 patterns and fills in their parameters; it never reaches further down.

Each layer in detail

L1	Data substrate	A relational time-series database storing one row per (date, instrument, field). Joined to an instrument master with vendor, ticker, asset class, curve family, country, currency, tenor. Every ingestion event logged with playbook hash, source-file hash, timestamp. <i>Finance-blind: knows trade dates and field values, does not know what a yield is.</i>
L2	Primitives	Single-purpose, type-safe, deterministic computations. Every primitive is isolated in its own module with a typed input and output schema, a pure compute function, and a methodology configuration file that declares every numeric and categorical convention with a value, a source tag, and a rationale. The LLM never invents primitives; humans do, with code review and unit tests.
L3	Operators	Strictly-typed transformations between canonical artifacts: series alignment, series arithmetic, threshold detection, event windowing, conditional aggregation, set selection, cross-sectional ranking. Operators know about <i>Series</i> , <i>SeriesSets</i> , <i>EventSets</i> , <i>Panels</i> , <i>WindowedPanels</i> , <i>Lineage</i> , and nothing about yields, spreads, or basis points. This is what makes the operator catalogue reusable across asset classes.
L4	Workflow substrate	A typed DAG validator and executor. Every workflow is loaded from declarative configuration, validated for acyclicity, slot-type coherence, unit compatibility, primitive resolver completeness, and archetype-signature presence, <i>all before</i> any compute runs. The executor walks the DAG in topological order, lifts primitive outputs into canonical artifacts via the bridge, and returns the terminal artifact in an envelope with a full lineage walk.
L5	Workflow templates	Parameterised workflows: the DAG topology is locked, but specific node parameters are exposed as a slot schema that the LLM router binds at runtime. Each template ships with an archetype signature, the structural cues the router matches against prompts. Variants of the same archetype with different aggregators or window semantics ship as separate templates.

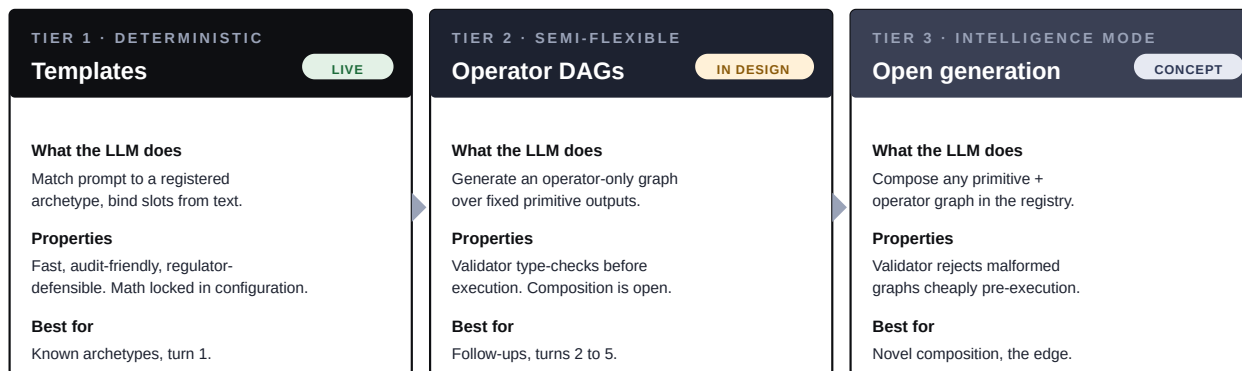
The bridge: how domain meets substrate

The bridge layer is the load-bearing piece that makes finance-blind operators useful. It does three things: it **lifts primitive outputs to canonical artifacts**, validating against the output schema, deriving the missingness policy from the methodology configuration, and writing a primitive-step lineage entry; it **enforces unit compatibility at validate-time**, so a basis-points series can only enter an operator that accepts basis points, or coerces cleanly via documented transitions; and it **drops back to a structured wire format on the way out**, emitting summaries with units, row counts, summary statistics and a full lineage walk.

The bridge is why the operator catalogue stays finance-blind: an operator gets a basis-points series and never asks what kind of basis points it is. It is why new primitives slot into the system with zero changes downstream. It is why methodology choices (rolling-window minimums, missingness policies, rounding) can change without invalidating the operator catalogue: the lineage chain records every choice, so cache invalidation and reproducibility audits work end-to-end.

05 The three-tier reasoning stack

Layered on top of the five substrate layers is the reasoning stack: how natural-language questions become executions. **Each tier relies on the closed substrate below.**



Three tiers, one validator. Every tier produces typed DAGs that the same substrate executes.

Why the substrate is the moat

A bare LLM with Bloomberg API access has no defensibility. Every malformed call becomes a runtime error, every numerical answer is unverifiable, and a single prompt change can produce a different number for the same question. There is nothing in such a system that an engineering team at a competitor cannot replicate in a week.

Macro Copilot inverts this. The closed primitive registry, the finance-blind operator catalogue, and the typed validation layer mean every LLM-generated graph is checked before it runs and every result is reproducible by hash. **Intelligence mode is not “let the LLM call APIs”. It is “let the LLM generate typed analysis graphs against a strongly-typed registry, validated before execution.”**

WHERE THE DEFENSIBILITY LIVES

The validator decides what the LLM is allowed to mean. The substrate decides what the LLM is allowed to compute. The moat is the registry and the validation, not the model behind the chat.

06 Reproducibility & lineage

Every artifact carries a **lineage**: an ordered list of steps ending in the most recent step that produced it. Each step is content-addressed, with a hash derived from the step name, version, parameters, and the hashes of all upstream steps.



terminal artifact hash = deterministic function of all upstream hashes

change any parameter, methodology, or input row, and the terminal hash changes; staleness is automatic

Every artifact is content-addressed. Reproducibility is a hash comparison.

Three implications that fall out for free

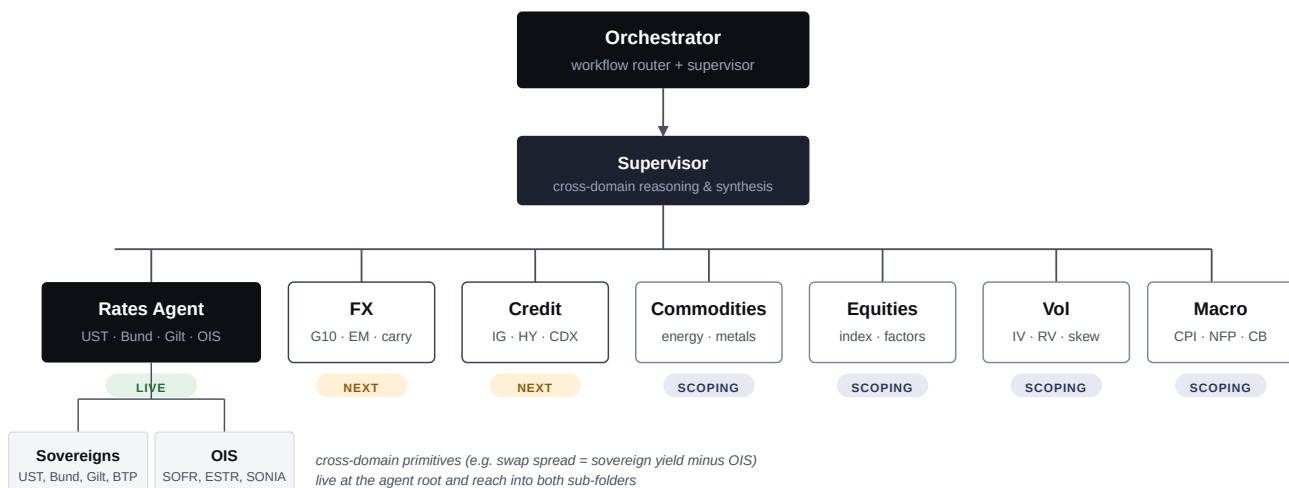
PROPERTY	MECHANISM
Equivalent computations have identical hashes	Re-running the same workflow with the same slot values against the same data produces a byte-identical lineage. Memoisation is free; reproducibility audits are a hash comparison.
Any artifact can be replayed	Given a lineage chain, the exact computation that produced an artifact months ago can be reconstructed. Every primitive's params, every operator's params, every input data hash, every config version is captured.
Cache invalidation is automatic	When a primitive's methodology changes (for example, increasing the minimum lookback window for a z-score from 60 to 90 days), every artifact whose lineage includes that primitive gets a new hash. Stale results are detectable; recomputation is targeted.

FOR THE AUDIT STORY

Every output the PM acts on traces back to source rows with full provenance. For regulators, risk, and post-mortems, "how did we get this number" collapses from a forensic exercise to a deterministic walk of the lineage chain.

07 Multi-agent architecture

Domain expertise is encapsulated in dedicated agents that each own their primitive registry, conventions, and routing logic. The orchestrator routes between them. Adding a new asset class is replication of a proven pattern, not a new architecture.



Each agent runs as an isolated tool server. Each owns its primitives, conventions and templates. The substrate runs them all unchanged.

The orchestrator routes; the supervisor synthesises; domain agents execute. Each domain repeats the same primitive pattern.

Routing flow for a user question

1. **Workflow router pre-gate.** The orchestrator checks every incoming turn against every registered template's archetype signature. If it matches, the template runs. Done in one LLM call plus deterministic execution.
2. **Supervisor fall-through.** For prompts that do not match a template, a tool-less supervisor LLM picks one or more domain children to delegate to. Each child runs in its own isolated tool server with its own tool access.
3. **Domain children execute.** Each child calls primitives (single-tool questions) or runs its own workflows (composed questions). Tools are deterministic; the LLM is only the dispatcher.
4. **Synthesiser.** Combines child outputs into a single answer with attribution back to each agent's contribution.

08 Worked example: a four-turn macro session

A real PM session usually takes the shape *open question* → *sharpen* → *condition* → *compare*. Below: a four-turn session with the routing tier, the substrate work, and what the PM sees on the screen at each step.

#	PM PROMPT	TIER	WHAT THE SUBSTRATE DOES
1	"Run an event study on the BTP–Bund 10Y spread when the 2Y OIS–Treasury spread widens by $> 1.5\sigma$."	TIER 1	Router matches the event-study archetype. Binds the signal instrument, target instrument, threshold, and post-event window. The substrate runs the 12-node DAG and returns a per-offset abnormal-return series.
2	"Now do that with the 5Y instead of the 10Y."	TIER 1	Router rebinds the target tenor only. Same DAG, same template, one slot changed. Cached lineage on every other node, so no recomputation outside the changed branch.
3	"Now also condition on whether the Fed is in a QT regime."	TIER 2	Prompt no longer matches the closed event-study archetype. The LLM emits an operator-only DAG that reuses the cached event slice and applies a regime mask via the threshold-detection operator on the QT signal series. The validator type-checks the graph before execution.
4	"Compare the QT-conditional move to last week's unconditional number."	TIER 2	One-operator combination over the two cached terminal artifacts via the series-arithmetic operator. No re-fetch, no re-compute. The result is itself a content-addressed artifact that can be saved as a view or referenced in turn five.

What the PM sees on screen

For every turn, the chat panel renders **(a)** the routing rationale (which template and tier was picked and why), **(b)** the slot bindings (what the LLM chose to fill into the template), **(c)** the terminal artifact (chart and summary statistics), and **(d)** the DAG lineage chain, with each node a clickable chip. Turn three's lineage chain visibly extends turn two's. Turn four's lineage chain references both turn one's and turn three's artifact hashes.

WHY THE SCREEN MATTERS

A trader rarely takes a copilot at its word on a single number. The lineage chips, the slot bindings, and the methodology block from each primitive's configuration file, all visible at one click, are how the system earns trust under the same scrutiny a senior analyst's spreadsheet gets.

The four turns above span Tier 1 and Tier 2 only. Tier 3 (open generation) lifts the same shape to questions for which no template or operator chain has been pre-validated. The LLM proposes a primitive and operator graph against the full registry, and the validator decides whether it runs.

09 Status & roadmap

The substrate is production-grade for rates today. The architectural pattern is proven; what remains is replication across asset classes and the conversational iteration loop on top.

Layer maturity

COMPONENT	STATUS	NOTES
L1–L4 · Substrate Data, primitives, operators, workflow executor	PRODUCTION	Rates feeds (Bloomberg + Refinitiv + ICE) wired in. ~10 rates primitives, 7 finance-blind operators, typed DAG validator + executor, full lineage. Substrate-level tests passing.
L5 · Templates (rates)	PRODUCTION	Two rates templates: an event-study archetype and a regime-conditioned relationship archetype. Realistic v1 catalogue: roughly 20 templates across 6 agents.
Frontend Workspace, copilot chat, library	PRODUCTION	Generic primitive workspace surface live. Conversational state visualisation is the next frontend phase.
Reasoning Tier 1 Template routing	PRODUCTION	Live for rates. Other agents inherit the pattern as they ship.
Reasoning Tier 2 Operator DAGs	NEXT PHASE	Substrate validator already supports it. Router needs a different prompt regime + operator-graph schema.
Reasoning Tier 3 Open generation	CONCEPT	Validator + bridge + lineage make it tractable. Ships once Tier 2 is validated in user testing.
Conversational iteration loop	NEXT PHASE	Substrate is ready. Orchestrator state machine (per-turn artifact cache, working-set, branching) is next.
Second domain agent FX or credit	SCOPING	Validates substrate portability beyond rates. Reuses the operator catalogue unchanged.

The two milestones in active development

MILESTONE 1 · THE CONVERSATIONAL ITERATION LOOP

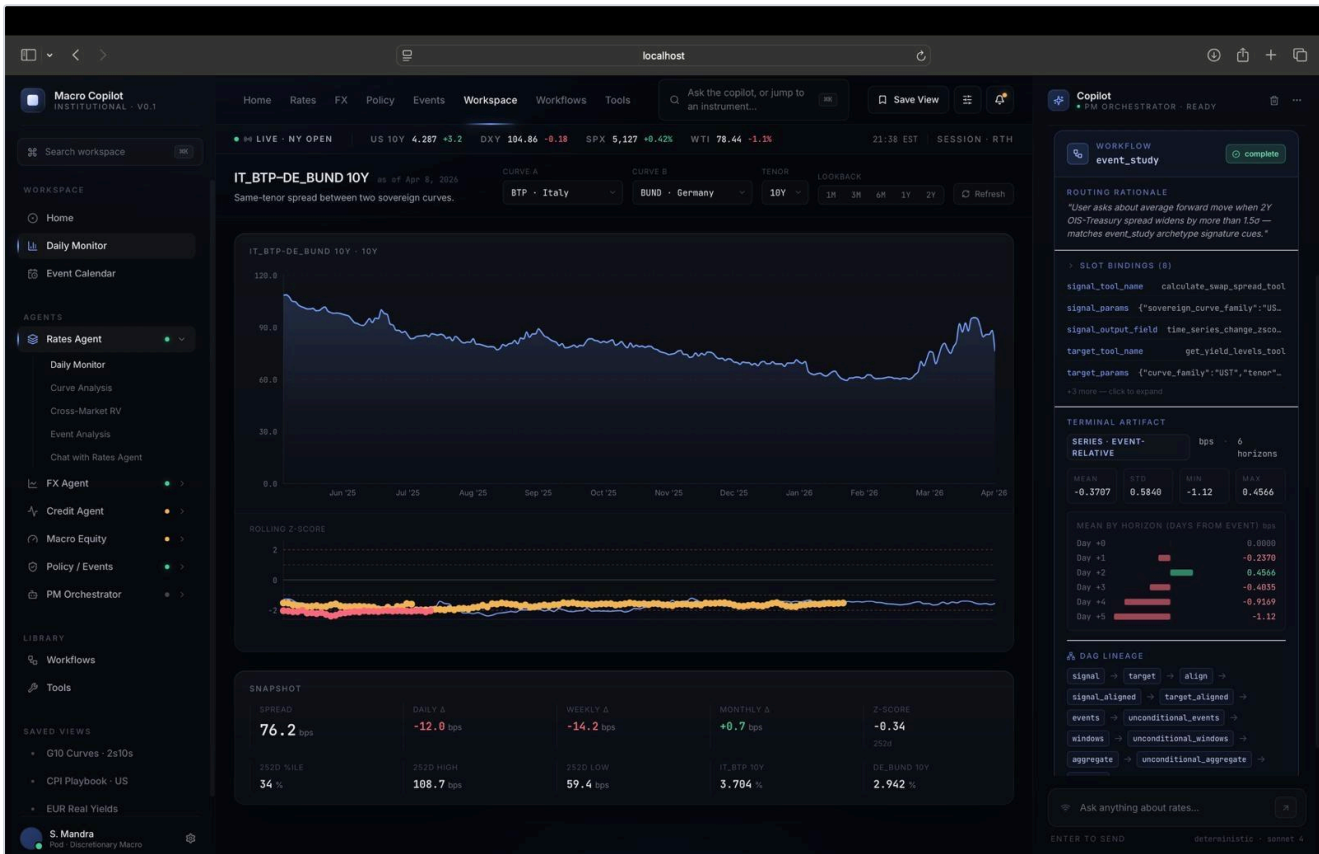
Per-turn workflow lineage carries forward. Follow-ups extend or mutate the prior DAG instead of restarting. The LLM router maintains a working set of artifacts addressable by reference. Branching forks conversational state.

MILESTONE 2 · THE SECOND DOMAIN AGENT

FX or credit. Same primitive pattern, same isolated tool-server architecture, same operator catalogue reused unchanged. The proof point that the substrate is genuinely cross-asset, not rates-shaped.

10 Product preview

Three surfaces sit on top of the typed primitive and operator registry: a streamed copilot chat, an interactive workspace for any registered primitive, and a catalogue of every tool and workflow in the system. Below: an event-study workflow run end-to-end on the BTP–Bund 10Y spread.

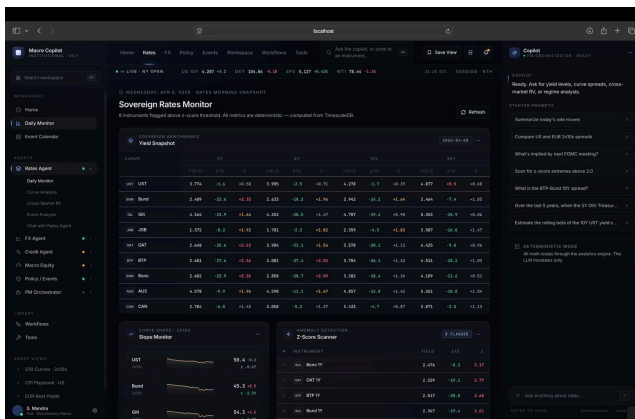


WORKSPACE AND WORKFLOW RESULT

The chart panel shows the BTP–Bund 10Y spread with its rolling z-score ribbon below. The right rail shows the executed workflow card: **routing rationale** (the LLM's explanation for why this template was picked), **slot bindings** (signal instrument, target instrument, parameters), **terminal artifact** (per-horizon mean, standard deviation, minimum, maximum, and a bar chart of mean by horizon in basis points), and the **DAG lineage** chain, with each node a clickable chip back to the primitive's methodology block.

This is the deterministic-mode payoff: the chart in the centre and the audit trail on the right tell the same story. The number on screen is reproducible by hash and explainable by inspection.

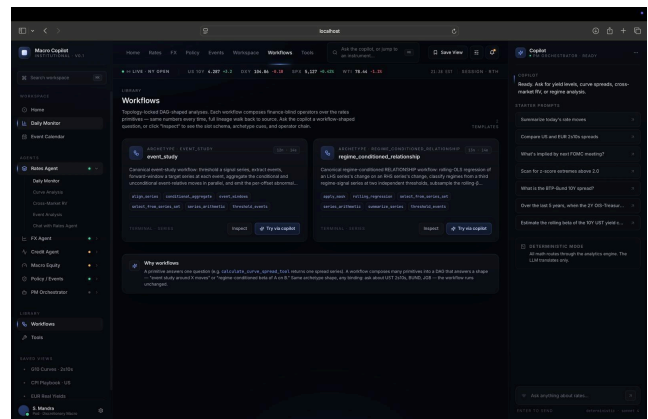
Daily monitor & workflows catalogue



DAILY MONITOR

The default rates surface: a yield snapshot across G10 sovereigns at 2Y, 5Y, 10Y and 30Y, a curve-shape monitor for 2s10s slope across countries, and an anomaly z-score scanner that flags instruments crossing methodology-defined thresholds. Every metric is computed deterministically from the time-series store and traceable back to source rows.

The starter prompts on the right rail ("*Compare US and EUR 2s10s spreads*", "*Estimate the rolling beta of the 10Y UST yield...*") are the bridge into the copilot chat. Click and the prompt drops into the composer, ready to send.



WORKFLOWS CATALOGUE

Every registered workflow template is human-readable. Each tile shows the archetype, a description, the operator chain (alignment, threshold detection, event windowing, conditional aggregation) and the terminal artifact type. *Inspect* opens the slot schema and the archetype-signature cues; *Try via copilot* drops a starter prompt into chat.

As new domain agents ship, their templates appear here automatically. The tools catalogue follows the same pattern at the primitive level: every registered primitive becomes immediately interactive in the workspace via a generic input form that is auto-built from the tool's input schema, with no per-primitive frontend code.

The frontend pattern in one sentence

Anything registered in the substrate appears in the UI. The PM never waits for an engineer to build a screen for a new primitive or workflow. The catalogue is generated from the registry, and the workspace renders any input schema as a controls rail with a Run button. **The substrate is the source of truth. The UI is a view of it.**

11 Why this architecture is the product

The architecture exists to deliver a single outcome: collapse the analyst’s research cycle from hours to minutes without giving up the rigor or auditability of work done in code. Each row below maps an architectural choice to the property it secures.

Strict typed DAG	Composition is safe. Type, unit, and shape mismatches surface at validate-time, not at runtime.
Closed primitive + operator registry	The LLM can only generate graphs whose components already exist in the registry. Anything outside the registry is rejected before it runs, so the failure mode is “no answer,” never “wrong answer.”
Content-addressed lineage	Reproducibility is automatic. Six-month-old answers re-run to byte-identical results.
Per-tool methodology configuration	Methodology is auditable and version-controlled. Every convention has a value, a source, and a rationale.
Bridge layer	Primitives stay domain-aware. Operators stay finance-blind. Both compose. New primitives slot in without changing the operator catalogue.
Multi-agent tree	Domain expertise is encapsulated. Cross-domain questions route correctly. Adding an agent is replication, not architecture.
Three-tier reasoning stack	Deterministic templates for known archetypes. Operator DAGs for follow-ups. Open generation for novel composition. One validator over all three.
Lineage chips + structured chat cards + workspace deep-links	The PM sees what the system <i>did</i> , not just what it <i>said</i> . Trust is earned by inspection, not asserted by branding.

IN ONE SENTENCE

Macro Copilot is not a chatbot for finance. It is a typed substrate for arbitrary macro analysis with an LLM-driven natural-language front-end. **The chat is the door. The substrate is the building.**

Contact

SREERAM ANDRA

Imperial College London
sreeram.andra25@imperial.ac.uk
[LinkedIn](#)

SACHA MIMOUN

Imperial College London
sacha.mimoun25@imperial.ac.uk
[LinkedIn](#)

PROJECT

Macro Copilot · v0.1
Research-stage prototype
Available for institutional review

This document is an architectural and product overview prepared for industry review. All system components described above are implemented or in active development per the maturity table on page 11. No proprietary information from any third party is reproduced in this document.